

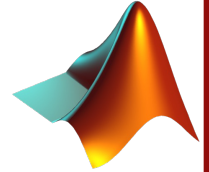
MATLAB® Tutorial: Control Statements and Plotting

Data Science and Decision Making (DSDM)

Indian Institute of Technology Gandhinagar, Gujarat



Program flow control

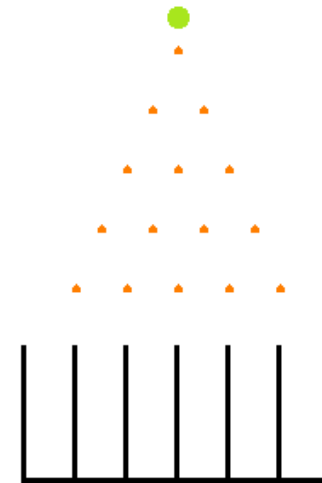
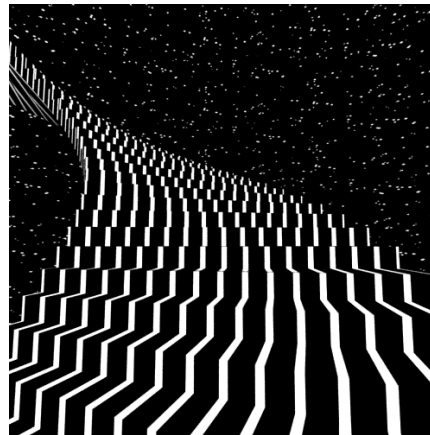


❑ Selection statements

- if statement
- if else statement
- Nested if statement
- switch statement

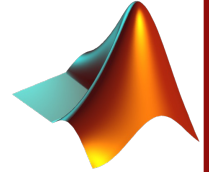
❑ Iterative statements

- for loop
- while loop
 - break statement
 - continue statement





Selection statements - if



❑ if statement

- Used to execute statements only if the condition is satisfied

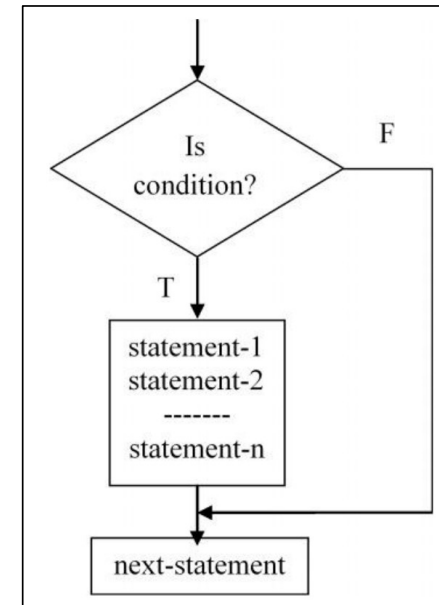
❑ Syntax

```
if (condition)
    statement_1
    statement_2
    :
    statement_n
end
```

❑ Example

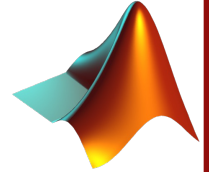
```
a = 10;
b = 20;

if(a < b)
    a = a + 99
end
```





Selection statements – if else



❑ if else statement

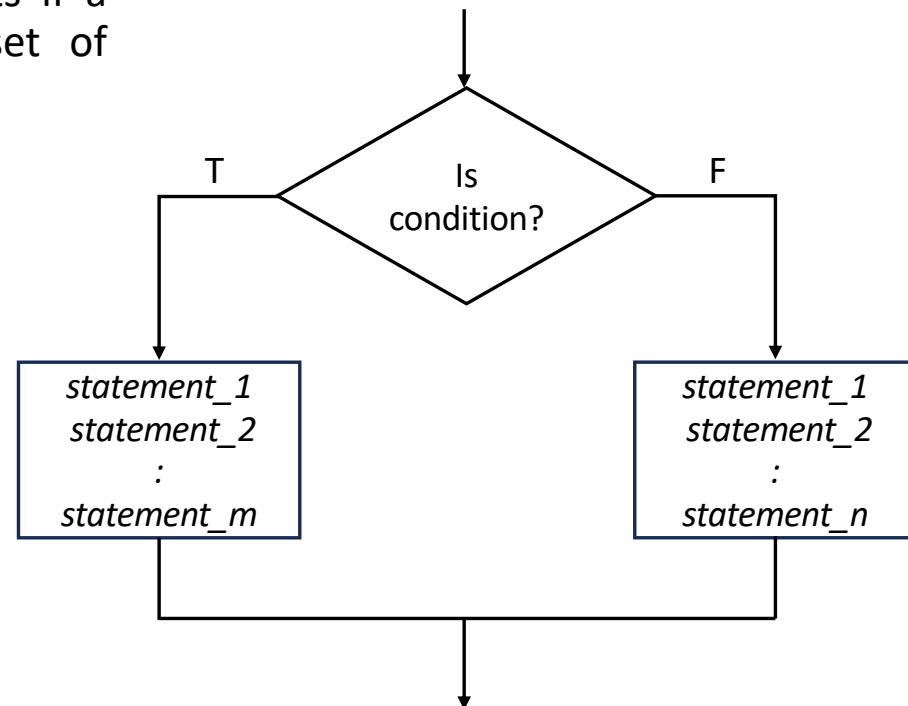
- Used to execute one set of statements if a condition is satisfied, and another set of statements, otherwise

❑ Syntax

```
if (condition)
    statement_1
    statement_2
    :
    statement_m
else
    statement_1
    statement_2
    :
    statement_n
end
```

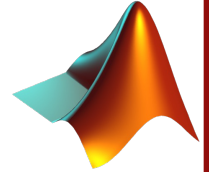
These are executed if the condition is TRUE

These are executed if the condition is FALSE





Selection statements – Nested `if`

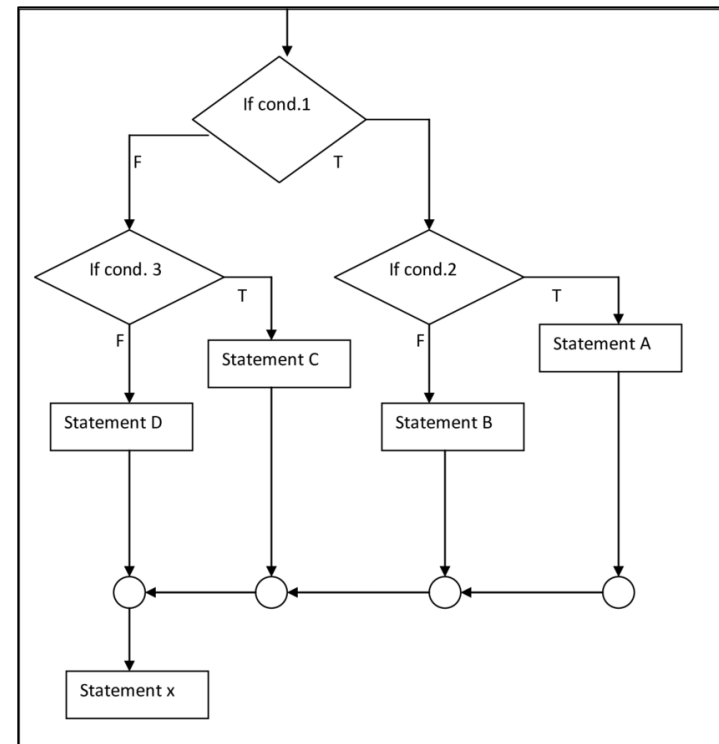


❑ Nested `if` statement

- if-else condition within if-else condition

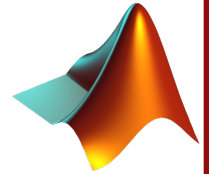
❑ Syntax

```
if(condition_1)
    statement_1
    :
    statement_m
elseif(condition_2)
    statement_1
    :
    statement_n
else
    statement_1
    :
    statement_p
end
```





Selection statements – switch



❑ switch statement

- Execute a particular set of statements based on the value of an expression

❑ Syntax

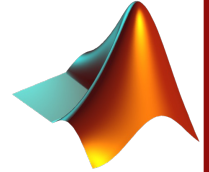
```
switch (switch_expression)
case case_expression
    statement_1
    :
    statement_m
case case_expression
    statement_1
    :
    statement_n
otherwise
    statement_1
    :
    statement_p
end
```

```
n = input('Enter a number: ');

switch n
    case -1
        disp('negative one')
    case 0
        disp('zero')
    case 1
        disp('positive one')
    otherwise
        disp('other value')
end
```



Iterative statements – while loop



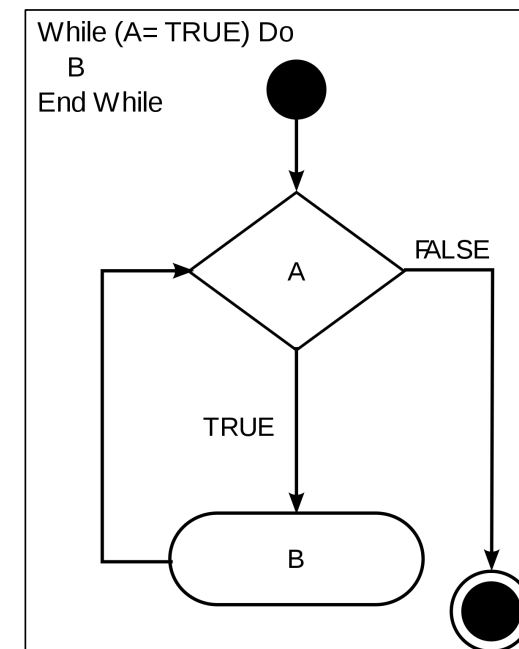
□ while loop

- while loops allow repeated execution of a code snippet as long as a specified condition is met

- Syntax:

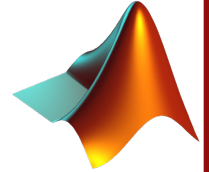
```
while(condition)
    statement_1 ...
    statement_n
end
```

- while loops are used when the number of iterations are not known beforehand
- It is important to ensure that the *condition* eventually becomes false to prevent infinite loop





Iterative statements – for loop

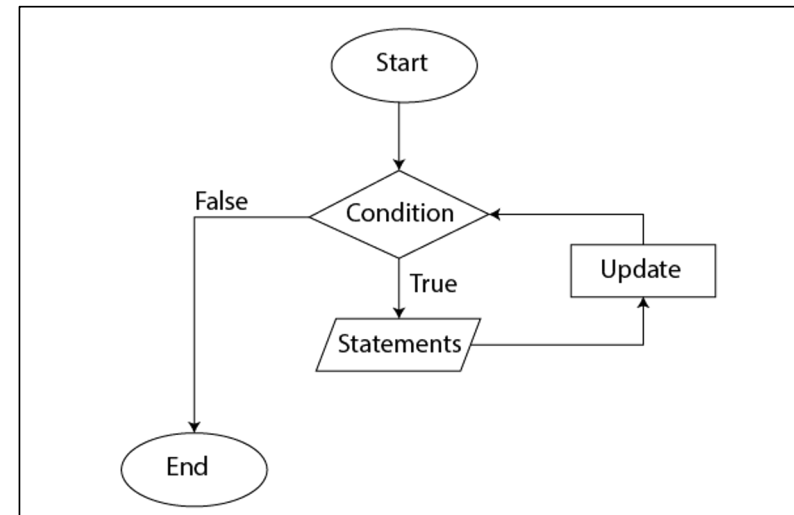


□ for loop

- for loops allow repeated execution of a code snippet over a specified number of times
- Syntax:

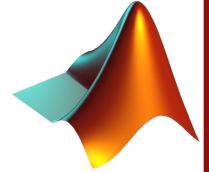
```
for index = values  
    statement_1 ...  
    statement_n  
end
```

- Here, *values* stores the data over which *index* iterates
- *values* can be a scalar or a vector
- It has to be ensured that *values* contains integers ≥ 1 , if *index* is to be used to access an array element





break and continue statements



❑ break statement

- A break statement is used to control the flow of the statement
- It is used to exit a loop prematurely, even if the loop condition is still true

❑ continue statement

- A continue statement is used to skip the current iteration of a loop and move to the next iteration
- Helps in increasing code efficiency by skipping unnecessary computations

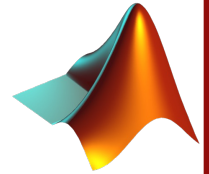
```
i = 1;
while(i < 10)
    disp(i);
    if(i == 2 | i == 5)
        continue;
    end
    if(i >= 7)
        break;
    end
end
```

Output:

```
1
3
4
6
```



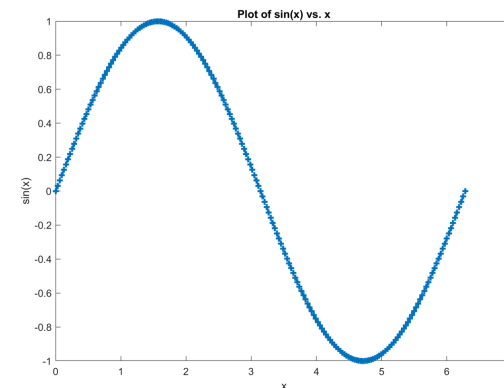
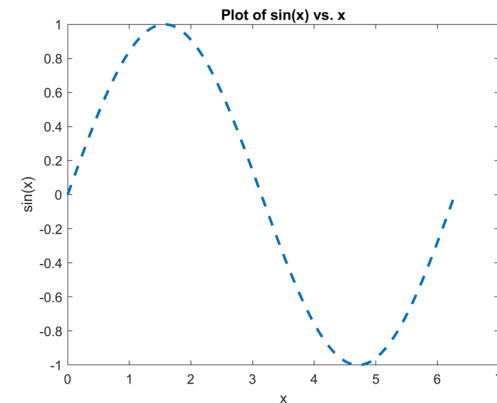
Plots in MATLAB®: 2-D Plots



❑ **Syntax:** `plot(xdata, ydata, options)`

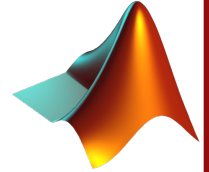
❑ **Line specifications**

- 'LineWidth': Change the thickness of the line
- 'LineStyle': Change the type of line plotted, to make it dotted, or solid, or a dash-dot line
 - E.g.: "-", "--", ":", "-."
- 'Marker': Add special symbols called markers on the curve
 - E.g.: "o", "+", "square", "."
- 'Color': Changes the color of the curve
 - E.g.: "r", "g", "b", "k", "m"



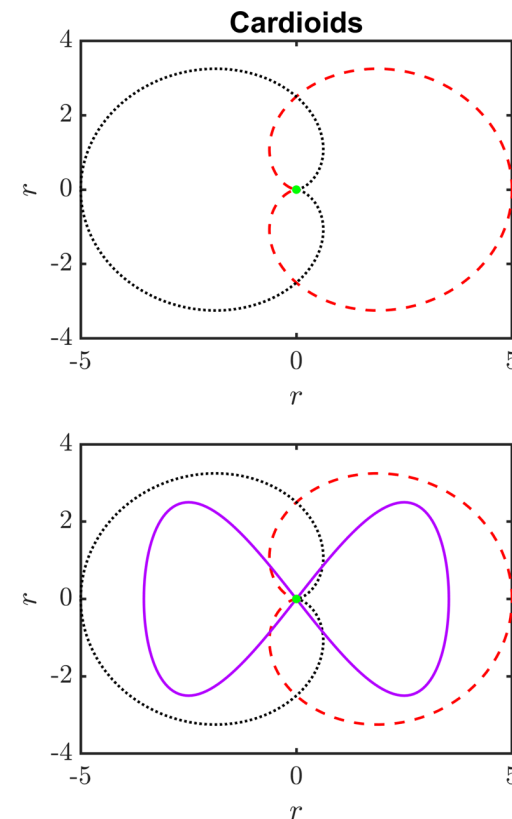


Plots in MATLAB®: 2-D Plots



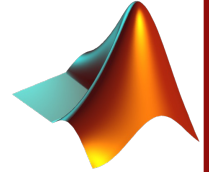
❑ Axes properties

- `xlabel`: Name the x-axis
- `ylabel`: Name the y-axis
- `xlim()`: Used to plot the graph within certain limits on the x-axis
- `ylim()`: Used to plot the graph within certain limits on the y-axis
- `gca`: Returns the current graphical axes
 - The properties of the axes can be modified using the `gca` command
 - `set()` function can be used to alter the properties of the axes object





Some other useful plots in MATLAB®



❑ **surf**(**X**,**Y**,**S**)

- Creates a 3D surface whose height **S** is a function of **X** and **Y**

❑ **mesh**(**X**,**Y**,**S**)

- Creates a 3D wireframe mesh

❑ **contour**(**X**,**Y**,**C**)

- Creates contour plot containing isolines of matrix **C**

❑ **streamline**(**X**,**Y**,**Z**,**U**,**V**,**W**,**xstart**,**ystart**,**zstart**)

- Creates 3D plot of streamlines that begin at **xstart**, **ystart**, and **zstart**

❑ **quiver**(**X**,**Y**,**U**,**V**)

- Plots arrows with directional components **U** and **V**

